

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	RAVURU PRANATHI
Name	192525076

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *R. Pranathi*

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Ans:

A Hadoop job is repeatedly failing because the input data is not divided properly among the mapper tasks. Some input splits are much larger than others. Because of this, some mapper tasks take a long time to finish, while other mappers finish quickly. The reducers have to wait for the remaining mapper outputs, so they remain idle for a long time.

The main design error is **uneven input partitioning**. This can happen when the input files have very different sizes or when the HDFS block size and InputSplit configuration are not suitable for the dataset. Another possible problem is an unsuitable partitioning method that sends too much data to one reducer.

In Hadoop, the input data is stored in **HDFS** and divided into blocks. MapReduce creates input splits from these blocks and assigns them to mapper tasks. Ideally, the splits should be reasonably balanced so that mapper tasks finish at nearly the same time.

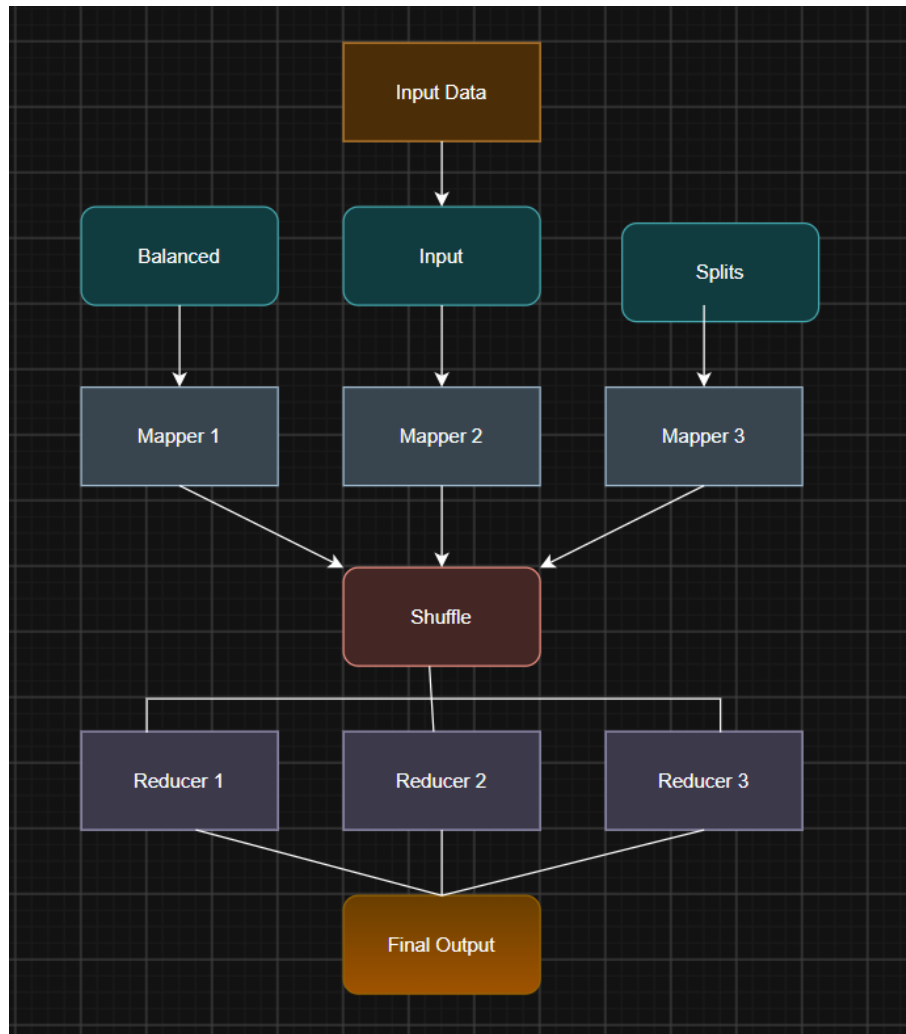
For example, suppose the input data is divided into three splits:

Split 1 → 10 GB

Split 2 → 2 GB

Split 3 → 2 GB

The mapper processing Split 1 will take much longer. The other mapper tasks may finish early, and the reducers will wait for the remaining output.



To correct the problem, the **HDFS block size and input split configuration** should be checked and adjusted according to the size and nature of the input data. The input files should be divided into reasonably balanced splits.

The number of mapper tasks should also be selected according to the available cluster resources. Creating too few mappers may increase processing time, while creating too many small tasks may increase task management overhead.

The reducer side also needs proper balancing. A **partitioner** should distribute keys evenly among reducers. If most records go to one reducer, that reducer becomes a bottleneck while the other reducers remain idle.

For example, if the key is a website URL and one URL occurs much more frequently than others, a normal partitioning method may send a large amount of data to one reducer. A suitable partitioning approach can help distribute the workload more evenly.

Data locality should also be considered. Mapper tasks should preferably process data on or near the DataNode where the data is stored. This reduces network traffic and improves processing speed.

The job should also be monitored to identify which mapper or reducer is taking unusually long. Hadoop counters and job monitoring can help identify skewed data and task imbalance.

Therefore, the main problem is **unbalanced input splits and reducer workload**. The solution is to use balanced input splits, suitable HDFS block and split settings, proper mapper allocation, an effective partitioner and data locality. These changes distribute the workload more evenly, reduce reducer idle time and improve the overall reliability and performance of the Hadoop MapReduce job.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Ans:

An HDFS system is designed to continue providing data even when one DataNode fails.

If data becomes unavailable after the failure of only one node, there is probably a problem with the replication configuration or NameNode high-availability design.

The first possible error is an insufficient replication factor. HDFS normally keeps multiple copies of each data block on different DataNodes. If the replication factor is set to 1, there is only one copy of a block. If the DataNode containing that copy fails, the block becomes unavailable.

For example:

Replication Factor = 1

Data Block A

↓

DataNode 1 ✗ Failed

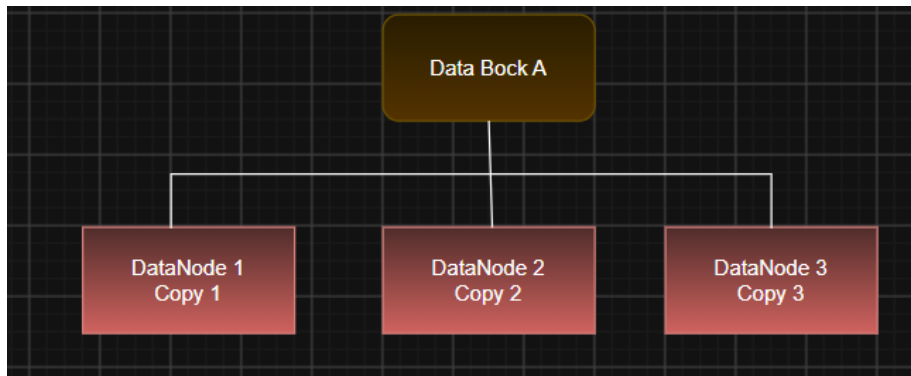
↓

No Copy

↓

Data Unavailable

For important data, a replication factor of 3 can be used:

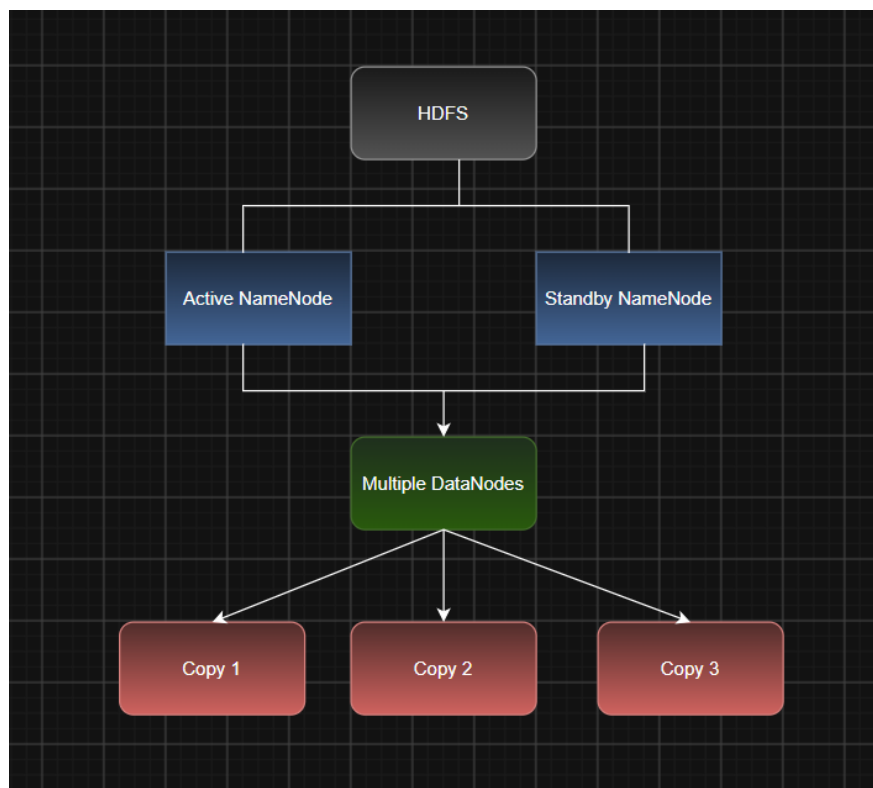


If one DataNode fails, another copy is still available. HDFS can also create a new replica on a healthy DataNode to restore the required replication level.

Another possible error is poor replica placement. If all replicas are stored on the same node or within the same rack, a failure in that location can affect all copies. HDFS should use rack-aware placement so that replicas are distributed across different racks where possible.

The second major problem may be the NameNode design. The NameNode maintains the metadata about files and their blocks. If the deployment has only one NameNode and it fails, the HDFS system may become unavailable even though the actual data blocks are still present on DataNodes.

A better design uses NameNode High Availability, with an Active NameNode and a Standby NameNode.



If the Active NameNode fails, the Standby NameNode can take over. This removes the NameNode as a single point of failure.

The system should also monitor DataNodes continuously. When a DataNode fails, HDFS should detect the failure and maintain the required number of replicas. Proper heartbeat monitoring helps the NameNode identify failed DataNodes.

Another important point is that replication alone cannot solve a NameNode failure. Similarly, NameNode high availability cannot protect against missing data blocks. **Both data replication and NameNode HA are required** for a reliable HDFS deployment.

For example, consider a video stored in HDFS. If its blocks have three replicas across different DataNodes and one DataNode fails, users can still access the video from the other replicas. If the Active NameNode also fails, the Standby NameNode can continue managing the HDFS metadata.

Therefore, the probable design errors are **low replication factor, poor replica placement, or a single NameNode without high availability**. The corrective solution is to use suitable HDFS replication, preferably three copies for important data, distribute replicas across nodes/racks, and configure Active–Standby NameNodes.

This design improves **fault tolerance, data availability and reliability** and prevents a single node failure from making important HDFS data unavailable.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Ans:

A data ingestion workflow uses ETL and Apache NiFi to move data into an analytics store. However, duplicate records are being created. This means the same record is being inserted more than once. The main problem is likely with repeated data ingestion, lack of unique identification, or incorrect NiFi/ETL configuration.

One common cause is that NiFi may process the same file or record more than once. This can happen when a processor is restarted or a failed transfer is retried without checking whether the record was already processed.

Another cause is the ETL process repeatedly reading the same source data. If the pipeline does not maintain a proper checkpoint or last processed value, it may load old records again during every execution.

For example, suppose the source database contains:

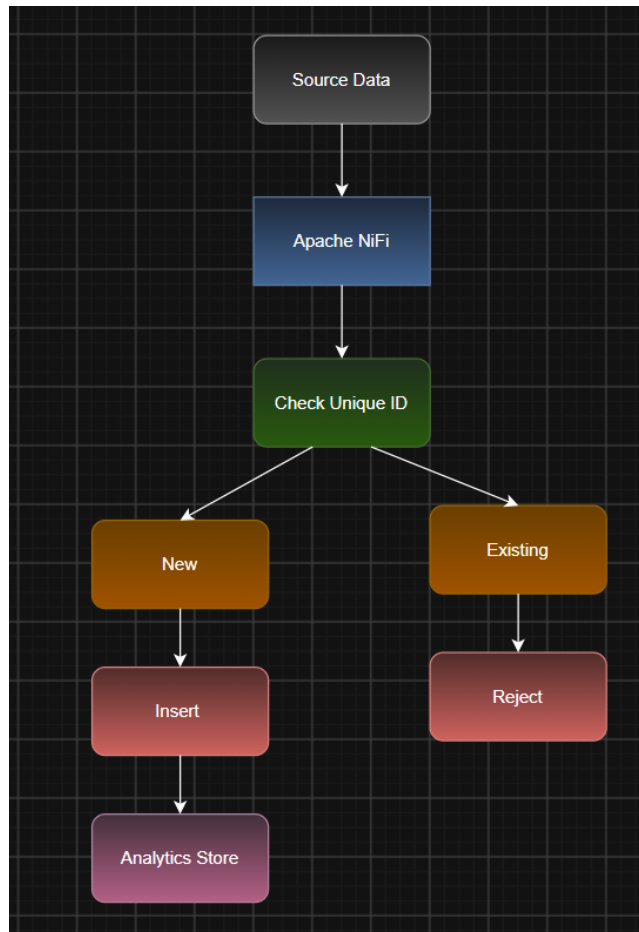
Transaction ID	Amount
T101	₹500
T102	₹700

If the ETL job reads the same records again during the next run without checking the previous load, the analytics store may contain:

T101	₹500	
T101	₹500	← Duplicate
T102	₹700	
T102	₹700	← Duplicate

The first corrective action is to use a unique ID such as Transaction ID, Customer ID or Order ID. The destination system should check this ID before inserting a new record.

A simple solution is:



NiFi should also be configured with proper state management and flow control so that already processed data is not unnecessarily processed again. Failed records can be sent to a retry or error queue instead of being blindly inserted again.

The ETL pipeline should use incremental loading rather than loading the complete dataset every time. For example, it can process only records added or modified after the previous successful execution.

Deduplication can also be performed before loading the analytics store. The destination database can enforce a primary key or unique constraint. Even if the same record reaches the destination twice, the second copy will not be stored.

Another important solution is maintaining a checkpoint. The pipeline can store the last successfully processed timestamp or record ID. During the next execution, only new records are selected.

For large datasets, duplicate records can also be identified using data comparison or hashing. Records with the same unique ID and content can be removed before final storage.

The main causes and fixes can be summarized as:

Likely Cause	Corrective Action
Same file processed repeatedly	Track processed files
ETL reloads old records	Use incremental loading
NiFi retry creates duplicates	Configure retry/state handling
No unique record ID	Use primary/unique key
No destination validation	Add duplicate checking
No checkpoint	Maintain last processed position

For example, in a banking system, if transaction T1001 is received twice because of a retry, the unique transaction ID allows the system to identify that T1001 already exists and prevents another copy from being inserted.

Therefore, the duplicate-record problem can be corrected by using unique identifiers, incremental ETL, NiFi state management, checkpoints, deduplication and destination-level constraints. These measures ensure that data is transferred reliably while preventing unnecessary duplicate records in the analytics store.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Ans:

The problem is that a YARN cluster is suffering from resource starvation when many users submit jobs at the same time. This means some jobs are getting most of the available CPU and memory, while other jobs are waiting for resources. The likely error is an improper YARN scheduling and resource allocation policy.

YARN uses the ResourceManager to manage cluster resources. If there are no proper queues, priorities or resource limits, a large job can consume most of the CPU and memory. Smaller jobs submitted by other users may then remain pending.

For example, suppose a cluster has 100 GB memory:

User 1 → Large Job → 80 GB

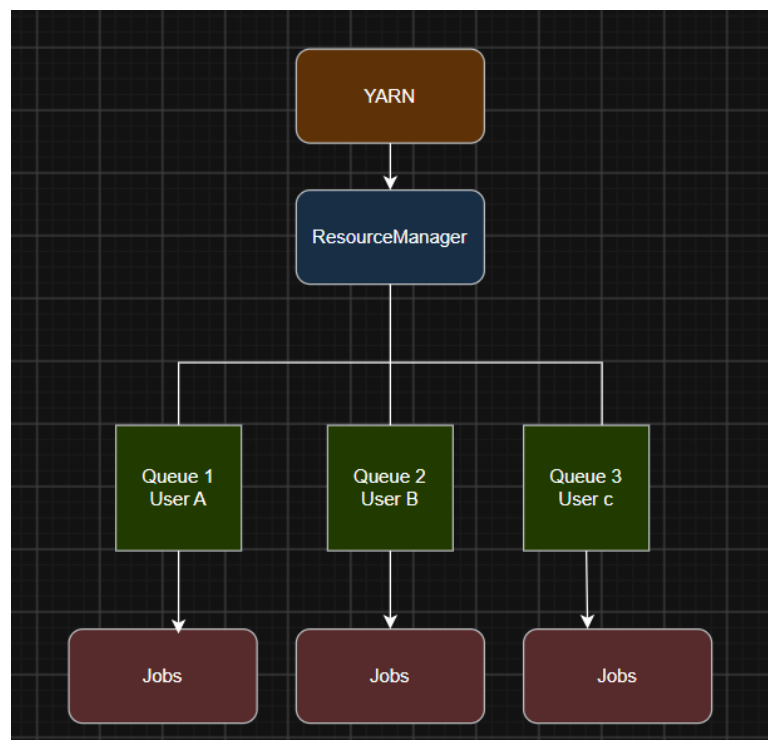
User 2 → Job → Waiting

User 3 → Job → Waiting

Here, User 1 is consuming most of the available resources, causing resource starvation.

The solution is to configure a suitable YARN scheduler such as the Capacity Scheduler or Fair Scheduler. These schedulers divide resources between users or groups and prevent one user from using the complete cluster.

A better design can be:



Each queue should be given a minimum resource capacity and a maximum resource limit. This ensures that every user receives a fair share of the cluster.

For example:

ETL Queue → 40%

Reporting Queue → 30%

ML Queue → 30%

The exact percentages can be changed according to the organization's requirements. If one queue is not using its resources, unused resources can temporarily be given to other queues.

Priority can also be used for important jobs. For example, an urgent ETL job can receive higher priority than a normal background job. However, priority should not allow one user to permanently consume all resources.

YARN should also use CPU and memory limits for individual applications. This prevents a single large job from consuming excessive resources.

Resource monitoring should be performed regularly. The administrator can check CPU, memory and queue usage and adjust the scheduler configuration when necessary.

If a node fails, YARN can detect the failure and reschedule the affected tasks on another available node. This improves resource availability and prevents a single node failure from stopping jobs completely.

Therefore, the main scheduling error is poor resource allocation without proper queues, limits and fair scheduling. Using the Capacity Scheduler or Fair Scheduler, resource quotas, queue priorities and monitoring provides better resource management.

In conclusion, proper YARN scheduling ensures that multiple users can run their jobs together without one user consuming all cluster resources. It improves fairness, resource utilization, job performance and overall cluster reliability.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

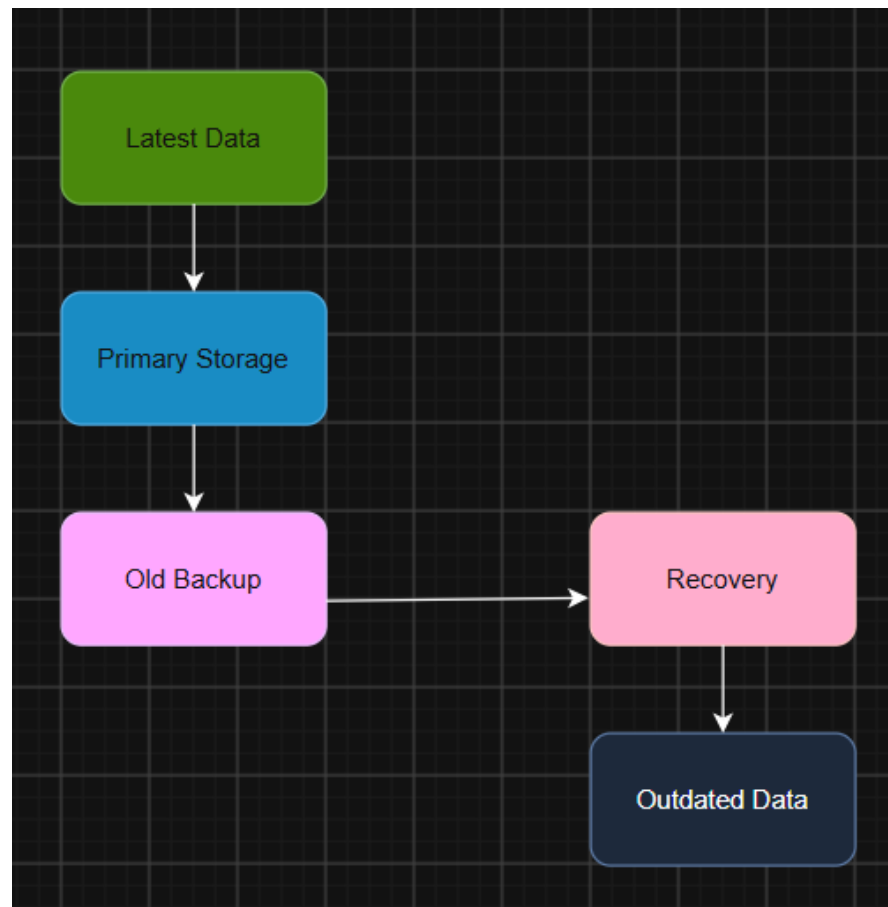
Ans:

The main problem is that the backup system is restoring old or outdated data after a failure. This indicates that the replication or backup process is not properly maintaining the latest version of the data.

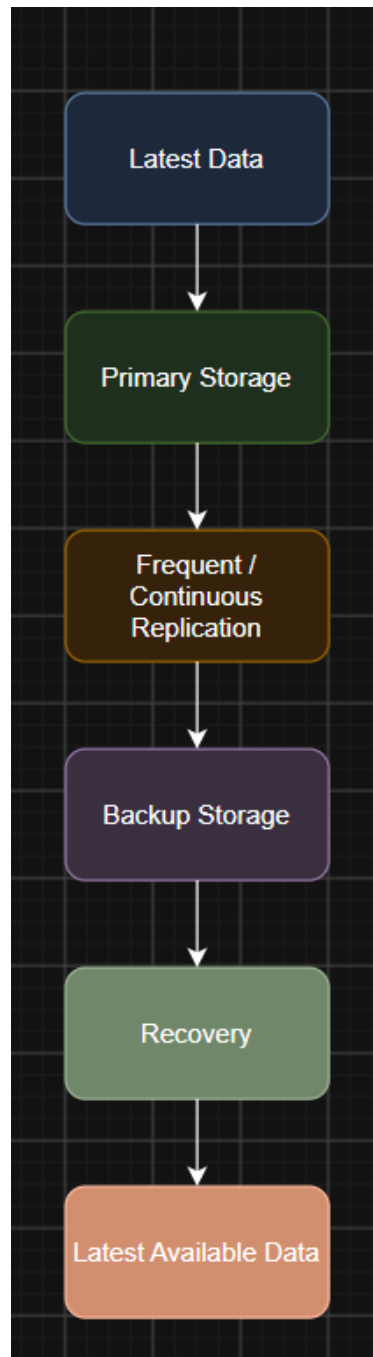
One probable design flaw is that backups are taken too infrequently. If a backup is created only once a day, any changes made after that backup will not be available during recovery.

Another possible problem is delayed replication. If data is replicated asynchronously, the latest changes may not have reached the backup storage when the main system fails. Therefore, recovery may use an older copy.

For example:



A better approach is:



The organization should use frequent or continuous backups depending on how important the data is. Critical data should be replicated more frequently so that the difference between the primary and backup copies is small.

The system should also maintain timestamps and version information for backups. During recovery, the administrator can identify and restore the most recent valid backup instead of selecting an older copy.

In a Hadoop environment, HDFS replication can provide multiple copies of data blocks across different DataNodes. However, replication and backup are not exactly the same. Replication mainly protects against

node failure, while backups or snapshots provide recovery from accidental deletion, corruption or other problems.

For example, if a file has three HDFS replicas and one DataNode fails, another replica can still be used. But if incorrect data is written to all three replicas, replication alone cannot recover the old correct version. A separate backup or snapshot strategy is therefore required.

Regular recovery testing should also be performed. This ensures that the backup actually contains usable and recent data.

Therefore, the probable design flaws are infrequent backups, delayed replication, lack of version tracking, or depending only on replication without a proper backup strategy. The solution is to use frequent/continuous replication, versioned backups or snapshots, proper timestamps and regular recovery testing.

This approach reduces data loss and ensures that recovery restores the most recent valid data instead of outdated information.